

Software Requirements Specification

for Mail Defender

Prepared by

Group Name: _____

Student	Student No.	Email
_____	_____	_____
_____	_____	_____
_____	_____	_____

Course: _____

Batch / Lab Section: _____

Teaching Guide: _____

Date of Submission: _____

Document Type: Software Requirements Specification

Version: 1.0

Status: Draft content prepared for college submission

Project: Mail Defender

Student, group, course, batch, guide, and date details are intentionally left blank so they can be filled later.

Contents

1. Introduction
 - 1.1 Document Purpose
 - 1.2 Product Scope
 - 1.3 Intended Audience and Document Overview
 - 1.4 Definitions, Acronyms and Abbreviations
 - 1.5 Document Conventions
 - 1.6 References and Acknowledgments
2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functionality
 - 2.3 Users and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 User Documentation
 - 2.7 Assumptions and Dependencies
3. Specific Requirements
 - 3.1 External Interface Requirements
 - 3.1.1 User Interfaces
 - 3.1.2 Hardware Interfaces
 - 3.1.3 Software Interfaces
 - 3.1.4 Communications Interfaces
 - 3.2 Functional Requirements
 - 3.3 Behaviour Requirements
 - 3.3.1 Use Case View
4. Other Non-functional Requirements
 - 4.1 Performance Requirements
 - 4.2 Safety and Security Requirements
 - 4.3 Software Quality Attributes
5. Other Requirements
 - Appendix A - Data Dictionary
 - Appendix B - Group Log

1. Introduction

Mail Defender is a web-based suspicious email analysis platform designed to help users inspect potentially unsafe emails before clicking links, opening attachments, replying to senders, or trusting the message content. The system receives raw email headers, email body text, and uploaded email files, then performs structured analysis using email parsing, sender authentication review, indicator extraction, threat intelligence lookups, local risk scoring, AI-assisted reasoning, and automated reporting.

This Software Requirements Specification describes the expected behavior, interfaces, constraints, performance needs, security expectations, and future upgrade path of Mail Defender. The document follows the structure of the provided SRS template and fills every required section with content specific to the Mail Defender project.

1.1 Document Purpose

The purpose of this SRS is to define the software requirements for Mail Defender, version 1.0. It specifies what the system must do, how users interact with it, what external systems it depends on, and what quality expectations must be satisfied. The document is intended to guide design, implementation, testing, evaluation, and future maintenance.

This SRS covers the Mail Defender web application, backend analysis service, threat intelligence integrations, report generation workflow, and future company-level features such as saved investigations, user accounts, organization workspaces, case management, and SIEM or ticketing integrations. The current scope focuses on single-email investigation, while future requirements describe how the system can evolve into an enterprise-level email threat investigation platform.

1.2 Product Scope

Mail Defender provides a single investigation workspace for suspicious email analysis. A user can paste raw headers and message body content or upload `.eml` and `.txt` files. The platform parses the email, extracts indicators of compromise, validates sender authentication signals, checks reputation intelligence, calculates a risk score, and generates a readable analyst-style report.

The product reduces manual phishing investigation effort by collecting multiple investigation steps into one workflow. Instead of separately checking headers, links, domains, IPs, hashes, and attachments across different tools, the user receives a structured verdict, evidence list, and recommended actions.

The main benefits are:

- Faster suspicious email triage.
- Better visibility into sender authentication and spoofing indicators.
- Structured IOC extraction from email content.
- Reputation checking through external threat intelligence services.
- Attachment hash calculation and file reputation support.
- AI-assisted summary and recommendations.
- Clean report generation for academic, helpdesk, SOC, or incident response use.
- A future roadmap toward company-level security operations.

1.3 Intended Audience and Document Overview

This document is intended for project evaluators, teaching guides, developers, testers, cybersecurity students, potential users, and future maintainers. The evaluator can use it to understand the project scope and completeness. Developers can use it to implement and maintain the system. Testers can derive test cases from the functional and non-functional requirements. Users and reviewers can understand what the system is expected to provide.

The document is organized as follows:

- Section 1 introduces the project, scope, readers, definitions, conventions, and references.
- Section 2 gives a high-level product description, including product perspective, user types, operating environment, constraints, documentation, assumptions, and dependencies.
- Section 3 explains specific requirements, external interfaces, functional requirements, and system behavior.
- Section 4 defines performance, safety, security, and quality attributes.
- Section 5 captures additional requirements useful for future company-level development.
- Appendix A provides a data dictionary.
- Appendix B provides a group log template.

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning
AbuseIPDB	A reputation service used to check whether an IP address has been reported for abusive behavior.
AI	Artificial Intelligence, used in this project to summarize evidence and recommend actions.
API	Application Programming Interface used for communication between software components.
BEC	Business Email Compromise, a fraud technique using impersonation and payment-related messages.
DKIM	DomainKeys Identified Mail, an email authentication method using cryptographic signatures.
DMARC	Domain-based Message Authentication, Reporting and Conformance policy for email authentication.
EML	A standard file format for storing an email message with headers and body.
IOC	Indicator of Compromise, such as a suspicious IP address, URL, domain, email address, or file hash.
SHA-256	A cryptographic hash algorithm used to uniquely represent attachment content.
SIEM	Security Information and Event Management system used for centralized security monitoring.
SOC	Security Operations Center responsible for monitoring and responding to security incidents.
SPF	Sender Policy Framework, an email authentication mechanism that checks allowed sending servers.
SRS	Software Requirements Specification.
VirusTotal	A threat intelligence platform used to check URLs, domains, IPs, and file hashes.

1.5 Document Conventions

This document uses numbered sections and subsections according to the provided SRS template. Requirement identifiers use the following prefixes:

Prefix	Meaning
FR	Functional Requirement
UI	User Interface Requirement
IR	Interface Requirement
PR	Performance Requirement
SR	Safety or Security Requirement
QA	Software Quality Attribute

The words **shall** and **must** indicate mandatory requirements. The word **should** indicates a recommended requirement. The word **may** indicates an optional or future enhancement. Priority values are classified as **High**, **Medium**, or **Future**.

This SRS uses a clean formal writing style suitable for college submission. Technical terms are defined in the glossary when needed. The document avoids unnecessary implementation detail unless it affects software requirements.

1.6 References and Acknowledgments

Reference	Description
Mail Defender Condensed Product Requirements Document	Primary product reference used to define scope, objectives, current features, target users, and future roadmap.
Provided SRS Template	Academic SRS structure used for section order and required content.
IEEE-style SRS practices	General requirements-documentation approach followed for clarity, traceability, and verifiability.
VirusTotal service documentation	Reference for reputation lookup concepts for URLs, domains, IPs, and file hashes.
AbuseIPDB service documentation	Reference for IP reputation and abuse confidence concepts.
Mail Defender project implementation	Current application behavior, frontend workflow, backend email analysis, risk scoring, and report generation.

Acknowledgment is given to the course faculty, teaching guide, cybersecurity learning resources, and open-source ecosystem that support defensive email investigation and educational security projects.

2. Overall Description

2.1 Product Perspective

Mail Defender is a new self-contained web application that can be used independently by cybersecurity students, IT helpdesk teams, SOC analysts, or incident response teams. It is not designed to replace a full email security gateway, but it complements existing email and security tools by providing a focused investigation workspace for suspicious messages.

The system sits between the user and external threat intelligence services. The user provides suspicious email evidence, the frontend sends the data to the backend analysis service, and the backend performs parsing, IOC extraction, authentication review, reputation checks, local scoring, AI-assisted synthesis, and report preparation.



Figure 1: High-level Mail Defender system context and component interaction

In future company-level upgrades, Mail Defender can connect with enterprise email platforms, SIEM tools, ticketing systems, organization-level case management modules, and admin dashboards. The core architecture is therefore designed around clear separation between user interface, analysis engine, external integrations, and reporting.

2.2 Product Functionality

At a high level, Mail Defender must let users submit suspicious emails, inspect extracted evidence, receive a risk verdict, and generate a clean report. The following functions summarize the system behavior; Section 3 expands them into detailed functional requirements.

- Accept suspicious email input through pasted headers/body and uploaded `.eml` or `.txt` files.
- Parse message headers and body into structured fields.
- Extract IOCs including IP addresses, URLs, domains, email addresses, and hashes.
- Extract attachments from email files and calculate SHA-256 hashes.
- Evaluate sender authentication signals such as SPF, DKIM, DMARC, reply-to mismatch, and spam status.
- Query threat intelligence sources such as VirusTotal and AbuseIPDB.
- Apply local phishing detection rules for urgency language, credential lures, suspicious top-level domains, shortened URLs, and reputation hits.
- Generate a risk score, verdict, attack type, key findings, analyst summary, and recommended actions.
- Display results in dashboards, evidence tables, and report sections.
- Allow copying and PDF exporting of the investigation report.
- Support future collaboration, case management, SIEM alerts, and organization-level administration.



Figure 2: Top-level data flow from suspicious email input to final report

2.3 Users and Characteristics

User Type	Characteristics	Primary Needs	Priority
Cybersecurity Student	Learning phishing analysis, email headers, and threat intelligence workflows.	Clear evidence, understandable report, academic presentation value.	High
SOC Analyst	Technically skilled user who investigates suspicious emails regularly.	Fast triage, IOC enrichment, case evidence, exportable reports.	High
IT Helpdesk Admin	Handles employee-reported suspicious emails and needs practical guidance.	Simple upload, quick verdict, user-safe recommendation.	High
Incident Response Team	Uses output as part of a larger investigation process.	Evidence preservation, hashes, indicators, severity, escalation notes.	Medium
Managed Security Provider	Future multi-client user requiring tenant separation and reporting.	Multi-tenant cases, dashboards, client reports.	Future
Employee Reporter	Non-technical user who reports suspicious emails.	Simple submission flow and safe next steps.	Future
System Admin	Responsible for configuration, users, integrations, and policies.	Access control, API keys, audit logs, rule management.	Future

The most important users for the first version are cybersecurity students, IT helpdesk administrators, and security analysts. Future versions should prioritize SOC managers, managed security providers, and enterprise admins.

2.4 Operating Environment

The current product operates as a web application. The frontend runs in a modern browser such as Chrome, Edge, Firefox, or Safari. The backend runs as an API service and uses environment variables for third-party API keys. The system can run locally during development and can later be deployed to a cloud server or enterprise environment.

Minimum development environment:

- A laptop or desktop computer.
- Node.js and npm for local development.
- A modern web browser.
- Internet connectivity for external reputation lookups.
- API keys for full VirusTotal, AbuseIPDB, and AI provider functionality.
- Sufficient memory to process normal email files and attachments.

Future production deployment should use HTTPS, a managed database, secure secret storage, background workers, logging, monitoring, and role-based access control.

2.5 Design and Implementation Constraints

ID	Constraint	Impact
C-01	The system depends on external threat intelligence APIs such as VirusTotal and AbuseIPDB.	API keys, rate limits, network latency, and provider downtime can affect result completeness.
C-02	Raw email content and attachments may contain sensitive or malicious data.	The system must handle content safely, avoid unnecessary exposure, and use privacy-aware logging.
C-03	The frontend must not expose secret API keys.	All sensitive integrations must be called through the backend.
C-04	The current MVP focuses on single-email analysis.	Bulk analysis and organization-wide monitoring are future enhancements.
C-05	Large email files and attachments can impact memory and processing time.	File size limits, validation, and later background processing are required.
C-06	AI-generated summaries may not always be perfect.	AI output must be treated as analyst assistance and grounded in structured evidence.
C-07	College submission requires a clean, understandable, and structured SRS.	The document must be readable for academic evaluators as well as technical reviewers.

2.6 User Documentation

The project should provide a setup guide, user guide, and short analyst guide. The setup guide should explain installation, environment variables, API keys, and how to start frontend and backend services. The user guide should explain how to paste email headers, upload `.eml` files, run analysis, read the score, inspect IOCs, and export reports. The analyst guide should explain how to interpret SPF, DKIM, DMARC, reply-to mismatch, threat intelligence results, hashes, and recommended actions.

2.7 Assumptions and Dependencies

The main assumptions and dependencies are:

- Users will provide valid email content or `.eml` files for analysis.
- Internet connectivity is available when external reputation lookups are enabled.
- VirusTotal, AbuseIPDB, and AI provider keys are configured for full functionality.
- The browser supports modern JavaScript and file upload APIs.
- Enterprise features such as user accounts and case management will require a database and authentication provider.
- The project is used for defensive email investigation and authorized security analysis.
- External API providers may change limits, response structures, or availability over time.
- Accurate verdicts depend on the quality of email evidence and threat intelligence results.

3. Specific Requirements

3.1 External Interface Requirements

Mail Defender communicates with users through a web interface and with external systems through backend API calls. The frontend must remain simple and responsive, while the backend must safely handle email content, secret keys, and threat intelligence integrations.

3.1.1 User Interfaces

The Mail Defender user interface is an investigation workspace. It must be simple enough for students and helpdesk users while still showing enough evidence for analysts. The current UI includes an input section, risk score card, authentication checks, header breakdown, IOC review board, attachment review, and formal report section.

ID	User Interface Requirement	Priority
UI-01	The system shall provide separate input areas for raw email headers and email body.	High
UI-02	The system shall support drag-and-drop and file picker upload for <code>.eml</code> and <code>.txt</code> files.	High
UI-03	The system shall show an analyzing state while backend analysis is running.	High
UI-04	The system shall display verdict, attack type, risk score, analyst summary, key findings, and recommendations.	High
UI-05	The system shall display SPF, DKIM, DMARC, reply-to mismatch, and spam status in a visually distinct verification panel.	High
UI-06	The system shall display extracted IOCs grouped by type with disposition tags such as Clean, Suspicious, Malicious, and Unknown.	High
UI-07	The system shall show attachment filename, content type, size, and SHA-256 hash when attachments are present.	High
UI-08	The system shall provide direct investigation links to VirusTotal for supported indicators and file hashes.	Medium
UI-09	The system shall allow users to reset the current investigation screen.	Medium
UI-10	The system shall allow users to copy or export the generated report.	High
UI-11	The system should support a light and dark theme.	Medium

3.1.2 Hardware Interfaces

Mail Defender is a web-based software system and does not require specialized hardware interfaces. It uses common computer hardware such as keyboard, mouse or touchpad, display, local storage access for file upload, and network adapter for API communication.

The application should run on a normal laptop or desktop used by students, analysts, or IT administrators. Future enterprise deployment may require server infrastructure, cloud hosting, load balancers, storage, and monitoring systems, but those are deployment infrastructure components rather than direct hardware interfaces for the user.

3.1.3 Software Interfaces

Software Component	Purpose	Data Exchanged
React Frontend	Provides web-based investigation interface.	Email headers, body, UI state, analysis result, report actions.
Backend API Service	Processes analysis requests and calls external services.	Email input, parsed metadata, IOCs, scores, enriched results.
Mail Parser Library	Parses raw email and attachments.	Headers, body, attachment metadata, attachment content.
VirusTotal API	Checks reputation of URLs, domains, IPs, and hashes.	Indicator value, type, detection statistics, disposition.
AbuseIPDB API	Checks IP address abuse reputation.	IP address, country, ISP, reports, abuse confidence score.
AI Analysis API	Generates summary and recommendations from structured evidence.	Evidence prompt, attack type, findings, actions, analyst summary.
Future Database	Stores users, reports, cases, settings, and audit logs.	Analysis history, cases, indicators, user metadata, policies.

3.1.4 Communications Interfaces

The frontend communicates with the backend over HTTP/HTTPS using JSON request and response payloads. The main analysis endpoint receives email headers and body content, then returns a structured JSON result containing verdict, score, authentication checks, header details, indicators, enrichments, attachments, and report-ready text.

The backend communicates with third-party APIs over HTTPS. All secret API keys must remain server-side. Future production deployment should require HTTPS for all user traffic, secure storage for API credentials, rate limiting for public endpoints, and logging that avoids unnecessary sensitive email body exposure.

3.2 Functional Requirements

The following functional requirements define the required behavior of Mail Defender. Requirements marked **High** are expected in the main product workflow. Requirements marked **Future** support company-level upgrades.

ID	Functional Requirement	Priority
FR-01	The system shall allow users to paste raw email headers and email body content into the investigation workspace.	High
FR-02	The system shall allow users to upload <code>.eml</code> and <code>.txt</code> files and automatically split headers and body where possible.	High
FR-03	The system shall parse common headers including From, Reply-To, Return-Path, Subject, Message-ID, X-Mailer, and originating IP.	High
FR-04	The system shall parse folded headers so multi-line header values are interpreted correctly.	Medium
FR-05	The system shall detect SPF, DKIM, and DMARC results from Authentication-Results headers.	High
FR-06	The system shall detect reply-to mismatch when the Reply-To domain differs from the visible sender domain.	High
FR-07	The system shall extract public IP addresses while excluding private, loopback, multicast, reserved, and date-like values.	High
FR-08	The system shall extract URLs, domains, email addresses, and supported hash values from headers and body content.	High
FR-09	The system shall normalize extracted domains for consistent analysis.	Medium
FR-10	The system shall limit the number of IOCs sent to external services to control performance and API usage.	Medium
FR-11	The system shall parse attachments from supported email files and compute SHA-256 hashes.	High
FR-12	The system shall add attachment hashes to the hash indicator list for reputation review.	High

ID	Functional Requirement	Priority
FR-13	The system shall query VirusTotal for supported URL, domain, IP, and hash indicators when configured.	High
FR-14	The system shall query AbuseIPDB for extracted public IP addresses when configured.	High
FR-15	The system shall continue analysis with partial results if an external provider fails or an API key is missing.	High
FR-16	The system shall calculate a phishing risk score between 0 and 100.	High
FR-17	The system shall map risk score to a verdict such as Safe, Suspicious, or High Risk.	High
FR-18	The system shall classify likely attack type such as Credential Harvesting, Business Email Compromise, Malware Delivery, Suspicious Email, or No Clear Attack Pattern.	Medium
FR-19	The system shall generate key findings and recommended actions based on evidence.	High
FR-20	The system shall generate an AI-assisted analyst summary when AI integration is configured.	Medium
FR-21	The system shall display all analysis results in a structured report view.	High
FR-22	The system shall allow users to copy the report to clipboard.	High
FR-23	The system shall allow users to export the report as a PDF.	High
FR-24	The system shall support saved analysis history in a future release.	Future
FR-25	The system shall support user accounts, roles, and organization workspaces in a future release.	Future
FR-26	The system shall support case management, comments, and analyst assignment in a future release.	Future
FR-27	The system shall support SIEM, SOAR, and ticketing integrations in a future release.	Future

3.3 Behaviour Requirements

The behavior of Mail Defender is centered on a suspicious email investigation workflow. The main actors are Employee Reporter, Security Analyst, and Admin. The MVP supports the analyst workflow, while employee submission and admin controls are planned for company-level upgrades.

3.3.1 Use Case View

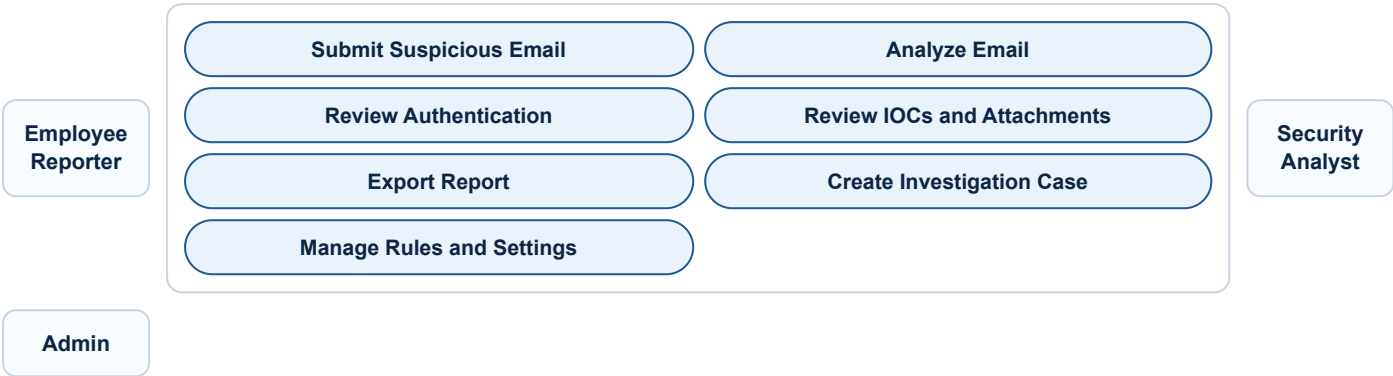


Figure 3: Use case view showing main actors and system goals

The use case diagram shows that users submit suspicious emails, analysts analyze evidence, review IOCs, export reports, and admins manage future rules and settings. These interactions are intentionally focused on defensive investigation and reporting.

Use Case Descriptions

Use Case	Actor	Short Description
Submit Suspicious Email	Employee Reporter or Analyst	The user uploads an <code>.eml</code> file or pastes raw header/body content into the workspace.
Analyze Email	Security Analyst	The system parses the email, extracts evidence, enriches indicators, scores risk, and returns a verdict.
Review Authentication	Security Analyst	The analyst reviews SPF, DKIM, DMARC, reply-to mismatch, and spam status.
Review IOCs and Attachments	Security Analyst	The analyst inspects URLs, domains, IPs, hashes, file metadata, and reputation results.
Export Report	Security Analyst or IT Admin	The user copies the report or exports it as a PDF for submission, documentation, or escalation.
Manage Rules and Settings	Admin	Future admin configures API keys, retention, users, roles, allowlists, blocklists, and detection rules.
Create Investigation Case	Security Analyst	Future analyst saves analysis as a case, assigns owner, adds comments, and tracks status.

Main Success Scenario

1. User opens the Mail Defender web application.
2. User uploads an `.eml` file or pastes raw email headers and body.
3. User clicks the Analyze Email action.
4. The frontend sends the email data to the backend API.
5. The backend parses the message, extracts IOCs, calculates attachment hashes, checks authentication, enriches indicators, and calculates risk score.
6. The system returns verdict, evidence, findings, recommendations, and report-ready content.
7. User reviews the dashboard and exports or copies the report.

Alternative and Exception Flows

- If the email has no clear header/body boundary, the system uses a best-effort split and still allows manual correction.
- If an API key is missing, the system marks the relevant enrichment unavailable and continues local analysis.
- If an external API fails, partial results are returned instead of failing the full investigation.
- If no suspicious signals are found, the system still recommends careful sender and context verification.
- If an attachment cannot be parsed, the system should continue analysis for available content and show an error or warning.
- If AI analysis is unavailable, local analysis remains the source of verdict and recommendations.

Behaviour Rules

- The system must never mark missing third-party evidence as clean evidence.
- The system must distinguish between clean, suspicious, malicious, and unknown dispositions.
- The system must keep evidence visible so users can understand why a verdict was assigned.
- The system must allow report generation only after analysis data exists.
- The system should prevent accidental duplicate analysis while one analysis is already running.

4. Other Non-functional Requirements

4.1 Performance Requirements

ID	Performance Requirement	Target
PR-01	The system shall load the main investigation interface quickly on a normal broadband connection.	Initial UI load should generally complete within 3 seconds after assets are cached.
PR-02	The system shall complete local parsing and IOC extraction without external APIs within a short time.	Local analysis should generally complete within 5 seconds for normal email size.
PR-03	The full analysis workflow may take longer when external APIs are enabled.	Common analysis should complete within 10 to 30 seconds depending on API latency.
PR-04	The user interface shall remain responsive while analysis is running.	Loading state must be visible and controls must not freeze.
PR-05	The backend shall reject or limit excessively large payloads to protect server memory.	Configured request size limit and validation must be enforced.
PR-06	The system should cache repeated IOC lookups in future releases.	Repeated indicators should not always require repeated external API calls.
PR-07	Future enterprise analysis should use background workers for long-running enrichment.	Bulk or sandbox processing must not block normal UI requests.
PR-08	Report export should complete without requiring manual formatting by the user.	PDF report generation should complete within a few seconds after analysis is available.

Performance depends on email size, attachment size, internet connectivity, and external provider response time. For academic evaluation, the MVP should demonstrate a complete workflow on sample suspicious emails.

4.2 Safety and Security Requirements

Because Mail Defender handles suspicious email content, security and privacy requirements are central to the product. The system must protect users from accidental exposure to malicious links or files and protect sensitive email content from unnecessary disclosure.

ID	Safety or Security Requirement	Rationale
SR-01	The frontend shall not execute scripts or active content from uploaded emails.	Prevents malicious HTML or script behavior during analysis.
SR-02	External links shall be displayed as evidence and opened only by deliberate user action.	Reduces accidental clicking of suspicious URLs.
SR-03	API keys shall be stored only on the backend or secure secret manager.	Prevents exposure of threat intelligence and AI credentials.
SR-04	Raw email content shall be treated as sensitive data.	Emails may contain personal, business, or confidential information.
SR-05	The system shall handle missing or failed integrations safely.	Security verdict should not silently imply clean status when data is unavailable.
SR-06	Future user accounts shall require authentication and role-based authorization.	Prevents unauthorized access to organization investigations.
SR-07	Future organization workspaces shall isolate tenant data.	Protects enterprise customers and managed service provider clients.
SR-08	Audit logs shall be added for future enterprise actions.	Supports accountability for analysis, rule changes, exports, and admin changes.
SR-09	The system should avoid storing raw email bodies unless storage is intentionally enabled.	Reduces privacy and data retention risk.
SR-10	The system should clearly indicate when AI or third-party services are used.	Helps users understand external data sharing and trust boundaries.

Expected Security Level

The MVP is expected to provide safe local investigation behavior and protect secret keys from frontend exposure. The company-level version is expected to provide authenticated access, role-based controls, secure storage, audit logs, tenant isolation, retention policies, and privacy controls for AI and third-party sharing.

4.3 Software Quality Attributes

4.3.1 Reliability

The system should return meaningful partial results even when some external integrations are unavailable. Local parsing, authentication checks, and local scoring should still work without third-party API keys. Error messages should be clear and actionable.

4.3.2 Usability

The interface should be understandable for cybersecurity students and IT users. Verdict, risk score, key findings, and recommendations should be easy to scan. Technical evidence should be available without overwhelming non-expert users.

4.3.3 Maintainability

The code should keep parsing, IOC extraction, enrichment, scoring, AI synthesis, and reporting as separate logical areas. Future developers should be able to add new rules or threat feeds without rewriting the complete application.

4.3.4 Portability

The application should run in standard web browsers and should be deployable on common server environments. Future production deployment should support cloud or self-hosted options.

4.3.5 Scalability

The MVP supports single-email analysis. Future enterprise versions should add database persistence, queues, caching, background workers, rate limiting, and horizontal scaling for team workloads.

4.3.6 Testability

Requirements should be testable through sample `.eml` files, unit tests for parsing and scoring, API tests for backend endpoints, and UI tests for upload, analysis, and report export workflows.

4.3.7 Interoperability

The system should integrate with external threat intelligence APIs today and future SIEM, SOAR, ticketing, email platform, and notification systems.

4.3.8 Robustness

The system should validate input, handle malformed emails, ignore invalid indicators, and avoid crashing when email fields are missing or third-party responses are incomplete.

5. Other Requirements

This section captures additional requirements that are useful for turning Mail Defender from an academic MVP into a company-level email threat investigation platform. These requirements are not all required for the first release, but they guide future upgrades.

Area	Requirement
Database	Future versions shall store users, organizations, analyses, reports, cases, indicators, attachments, settings, and audit logs in a persistent database.
Retention	Administrators should configure how long raw emails, reports, logs, and derived indicators are retained.
Privacy	Organizations should control whether raw email content is sent to AI providers or only redacted summaries are sent.
Internationalization	The initial version may use English, while future versions may support additional languages for employee guidance and reports.
Legal and Ethical Use	The system shall be used only for defensive email security analysis, education, and authorized investigation.
Reporting	Reports should include generated date, verdict, attack type, score, evidence, and recommendations.
Future Integrations	The product may integrate with Microsoft 365, Google Workspace, SIEM tools, ticketing systems, Slack, Teams, and browser extensions.
Future Analytics	Company dashboards should show analysis volume, common attack types, top malicious domains, response status, and high-risk trends.

Future Roadmap Summary

1. Stabilize the MVP with validation, tests, sample emails, and improved error handling.
2. Add persistent analysis history with search and filtering.
3. Add authentication, organization workspace, and role-based access control.
4. Add case management with comments, assignment, status, and report history.
5. Add SIEM, SOAR, ticketing, and email platform integrations.
6. Add advanced detection such as URL redirect analysis, domain age checks, sandboxing, and configurable rules.
7. Prepare enterprise readiness with SSO, audit logs, retention policies, tenant isolation, and compliance exports.

Appendix A - Data Dictionary

The data dictionary lists important data items, state variables, constants, inputs, outputs, and related operations used in Mail Defender. This helps developers and testers understand how information flows through the system.

Name	Type	Description	Related Requirement
emailHeaders	Input String	Raw email header block pasted or extracted from uploaded file.	FR-01, FR-02, FR-03
emailBody	Input String	Email body text or HTML-derived content used for phishing analysis.	FR-01, FR-02, FR-08
uploadedFile	File	.eml or .txt file supplied by the user.	FR-02
fromHeader	String	Visible sender value extracted from headers.	FR-03, FR-06
replyToHeader	String	Reply-To value used to detect reply mismatch.	FR-03, FR-06
returnPath	String	Return-Path header used as sender/routing evidence.	FR-03
subject	String	Email subject line used for social engineering signals.	FR-03, FR-16
messageId	String	Message-ID header extracted for evidence.	FR-03
authChecks	Object Array	SPF, DKIM, DMARC, reply-to mismatch, and spam status results.	FR-05, FR-06
spfStatus	Enum	Possible values include pass, fail, softfail, neutral, none, unknown, and error.	FR-05
dkimStatus	Enum	Possible values include pass, fail, none, unknown, and error.	FR-05
dmARCStatus	Enum	Possible values include pass, fail, quarantine, reject, none, and unknown.	FR-05
iocIps	Array	Public IPv4 indicators extracted from headers and body.	FR-07
iocUrls	Array	HTTP and HTTPS URLs extracted from email content.	FR-08

Appendix A - Data Dictionary Continued

Name	Type	Description	Related Requirement
iocDomains	Array	Domains extracted from URLs, email addresses, and raw text.	FR-08, FR-09
iocEmails	Array	Email addresses extracted from message content.	FR-08
iocHashes	Array	MD5, SHA-1, SHA-256, and computed attachment hashes.	FR-08, FR-11, FR-12
attachmentDetails	Object Array	Filename, content type, size, and SHA-256 hash for each attachment.	FR-11
virusTotalReport	Object Array	Reputation response for URL, domain, IP, or hash indicators.	FR-13
abuseIpDbReport	Object Array	IP reputation response including reports and abuse confidence score.	FR-14
riskScore	Integer	Calculated phishing score between 0 and 100.	FR-16
verdict	Enum	Possible values are Safe, Suspicious, and High Risk.	FR-17
attackType	Enum	Credential Harvesting, Business Email Compromise, Malware Delivery, Suspicious Email, or No Clear Attack Pattern.	FR-18
keyFindings	Array	Short evidence-based findings shown in the report.	FR-19
recommendedActions	Array	Actionable next steps for the user or analyst.	FR-19
analystSummary	String	AI-assisted or local summary explaining the overall assessment.	FR-20
reportPdf	File Output	Generated PDF report for documentation or submission.	FR-23
analysisSource	Object	Indicates whether local rules and AI were used.	FR-16, FR-20

Requirement Traceability Snapshot

Feature Area	Main Requirements	Test Approach
Email Intake	FR-01 to FR-02, UI-01 to UI-03	Paste text and upload sample <code>.eml</code> and <code>.txt</code> files.
Parsing and IOC Extraction	FR-03 to FR-10	Use sample emails containing URLs, domains, IPs, hashes, and folded headers.
Threat Enrichment	FR-13 to FR-15	Test valid keys, missing keys, failed providers, and unknown indicators.
Risk and Report	FR-16 to FR-23	Verify verdict mapping, findings, recommendations, copy report, and PDF export.
Enterprise Upgrade	FR-24 to FR-27	Validate through future design, prototype, or implementation tests.

Appendix B - Group Log

This section is reserved for group meeting minutes, task distribution, contribution records, and other activity details required by the teaching assistant or project evaluator. The following table is intentionally left as a fillable structure.

Date	Members Present	Activity / Discussion	Outcome / Next Step

Final Conclusion

Mail Defender is a modern phishing investigation and email threat analysis platform. Its strength lies in combining email parsing, sender authentication review, IOC extraction, threat intelligence enrichment, AI-assisted reasoning, local risk scoring, and automated reporting into one structured workflow.

The current MVP is suitable for academic demonstration because it clearly shows a real cybersecurity use case with practical analysis output. The planned roadmap can evolve it into a company-level investigation platform for SOC teams, enterprise security teams, and managed security providers.

Prepared Content Note: This SRS content was drafted from the Mail Defender PRD reference and the provided SRS template. Student, group, course, and guide details should be added before final submission.